BEGINNER TO PRACTICAL MASTERY

C++ Arrays

A Complete, Professional Guide — Concepts, Applications, Notes & 10 Fully-Explained Practical Examples

```
int numbers[5] = { 10, 20, 30, 40, 50 };
```

STRUCTURED • BEGINNER-FRIENDLY • EXAM & INTERVIEW READY

Table of Contents

1. What is an Array? 3
2. Why Arrays Matter — Real-World Applications 4
3. How Arrays Are Stored in Memory 5
4. Declaring and Initializing Arrays 6
5. Accessing and Modifying Elements 7
6. Arrays and Functions 8
7. Multidimensional Arrays 9
8. Arrays vs Pointers vs std::array vs std::vector 10
9. Common Pitfalls & Best Practices 11
10. 10 Practical Examples (Full Syntax + Explanation) 12
11. Quick Reference Cheat Sheet 22

1. What is an Array?

An **array** is a fixed-size, ordered collection of elements that all share the **same data type**, stored in **contiguous (back-to-back) memory locations**. Instead of creating separate variables like `score1`, `score2`, `score3`, you create a single array that holds all of them together, and you access each one using an **index** (a position number).

```
int scores[5] = {90, 85, 76, 99, 60};
// scores[0] = 90, scores[1] = 85, scores[2] = 76, scores[3] = 99, scores[4] = 60
```

Key characteristics of a C++ array:

- **Fixed size** — once declared, a plain C-style array cannot grow or shrink.
- **Homogeneous** — every element must be of the same type (all `int`, all `double`, etc.).
- **Zero-indexed** — the first element is at index `0`, not `1`.
- **Contiguous memory** — elements sit one right after another, which makes access extremely fast.
- **Direct memory access** — the array name itself behaves like a pointer to its first element.

> **Analogy for Beginners**
> Think of an array like a row of numbered lockers in a school hallway. Every locker is the same size (same type), they are placed side by side (contiguous memory), and each locker has a number starting from 0 (the index) that lets you go straight to it without opening every locker before it.

2. Why Arrays Matter — Applications

Arrays are one of the most fundamental building blocks in programming. Understanding where they are used helps beginners see why this topic deserves so much attention.

Application Area	How Arrays Are Used
Data storage	Storing a fixed collection such as student marks, temperatures, or sensor readings.
Mathematical computation	Vectors, matrices, and mathematical grids (2D/3D arrays) for scientific computing and graphics.
Searching & Sorting	Core algorithms like linear search, binary search, bubble sort, and quicksort operate directly on arrays.
Image processing	An image is essentially a 2D (or 3D for color) array of pixel values.
Game development	Game boards (chess, tic-tac-toe), tile maps, and inventories are represented as arrays.
Buffers & I/O	Reading data from files or networks into fixed-size character/byte buffers.
Lookup tables	Precomputed values (like factorials or hash tables) stored for instant retrieval instead of recalculating.
Data structures	Arrays are the foundation for stacks, queues, heaps, and hash tables implemented from scratch.
String handling	C-style strings are literally character arrays terminated by a null character `'\0'`.
Embedded systems	Fixed memory devices rely heavily on arrays because dynamic allocation is often avoided.

> **Why This Matters for Interviews**
>
> Arrays are almost always the very first topic in coding interviews and computer science courses because nearly every other data structure (vectors, stacks, queues, hash maps, matrices) is either built on top of an array or was inspired by one.

3. How Arrays Are Stored in Memory

When you declare `int arr[5];`, the compiler reserves one continuous block of memory large enough for 5 integers. If an `int` takes 4 bytes, the array takes `5 × 4 = 20` bytes total, laid out back-to-back.

```
Index:      0     1     2     3     4
Value:     10    20    30    40    50
Address: 1000  1004  1008  1012  1016   // assuming int = 4 bytes
```

This contiguous layout is why arrays support **constant-time O(1) access**: to find element `arr[i]`, the compiler simply computes:

```
address_of(arr[i]) = base_address + (i * size_of_each_element)
```

> **Note**
> This is exactly why array indexing is so fast — there is no searching involved. The CPU jumps directly to the calculated memory address.

4. Declaring and Initializing Arrays

4.1 Basic Declaration

```
dataType arrayName[arraySize];

// Examples
int age[10];        // array of 10 integers, values are garbage until set
double price[4];    // array of 4 doubles
char grade[26];     // array of 26 characters
```

4.2 Declaration with Initialization

```
int nums[5] = {1, 2, 3, 4, 5};      // fully initialized
int nums2[5] = {1, 2};              // remaining elements become 0 -> {1,2,0,0,0}
int nums3[] = {1, 2, 3, 4};         // size deduced automatically as 4
int zeros[100] = {0};               // all 100 elements set to 0
int zeros2[100]{};                  // C++11 uniform init, all elements = 0
```

Common Mistake

`int arr[5]; arr = {1,2,3,4,5};` — this does NOT compile. You can only use brace-initialization at the moment of declaration, not as a later assignment.

4.3 Array Size Must Be Known

For a plain C-style array, the size must be a **compile-time constant** (a fixed number or a `const` known at compile time):

```
const int SIZE = 10;
int data[SIZE];      // valid, SIZE is a compile-time constant

int n;
std::cin >> n;
int data2[n];       // NOT standard C++ (VLA) — avoid; use std::vector instead
```

5. Accessing and Modifying Elements

```cpp
int marks[5] = {70, 85, 90, 60, 75};

std::cout << marks[0];      // prints 70 (first element)
std::cout << marks[4];      // prints 75 (last element)

marks[2] = 100;             // modify the 3rd element
marks[0] = marks[0] + 5;    // update using its own value
```

Out-of-Bounds Access — Undefined Behavior

`marks[5]` does not exist (valid indices are 0–4), yet C++ will **not stop you** from writing `marks[5] = 10;`. This silently corrupts nearby memory and is one of the most common sources of bugs and security vulnerabilities in C++ programs. Always double-check your bounds.

5.1 Traversing an Array (Visiting Every Element)

```cpp
// Classic for-loop
for (int i = 0; i < 5; i++) {
    std::cout << marks[i] << " ";
}

// Range-based for-loop (C++11 and later) — cleaner, safer
for (int m : marks) {
    std::cout << m << " ";
}
```

6. Arrays and Functions

When an array is passed to a function, C++ does **not** copy the whole array — it "decays" into a pointer to its first element. This means the function can modify the original array, and it also means the function loses information about the array's size, so the size must be passed separately.

```cpp
// Function that receives an array (decays to a pointer)
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
}

// Equivalent, more explicit pointer syntax
void printArray2(int* arr, int size) { /* same as above */ }

int main() {
    int data[4] = {5, 10, 15, 20};
    printArray(data, 4);   // pass the array and its size
}
```

Note

Because the array decays to a pointer, `sizeof(arr)` **inside** the function returns the size of a pointer (commonly 8 bytes), not the size of the array. Always pass the length explicitly, or prefer `std::array` / `std::vector`, which carry their size with them.

7. Multidimensional Arrays

A 2D array is essentially an "array of arrays" — commonly used to represent tables, grids, or matrices.

```
int matrix[3][4];  // 3 rows, 4 columns

int grid[2][3] = {
    {1, 2, 3},
    {4, 5, 6}
};

std::cout << grid[1][2];  // prints 6 (row index 1, column index 2)

for (int r = 0; r < 2; r++) {
    for (int c = 0; c < 3; c++) {
        std::cout << grid[r][c] << " ";
    }
    std::cout << "\n";
}
```

Arrays can also have three or more dimensions, e.g. `int cube[2][3][4];`, though these are used less frequently and mostly appear in scientific computing or 3D graphics.

8. Arrays vs Pointers vs std::array vs std::vector

Feature	C-style array	std::array (C++11)	std::vector
Size	Fixed, compile-time	Fixed, compile-time	Dynamic, can grow/shrink
Knows its own size	No	Yes (`.size()`)	Yes (`.size()`)
Bounds checking	None	Optional via `.at()`	Optional via `.at()`
Memory location	Stack (usually)	Stack (usually)	Heap
Recommended for	Legacy / low-level code	Fixed-size modern C++	Most general-purpose use

Best Practice for Beginners

Learn plain C-style arrays first (this guide) because they explain how memory really works — but in modern, day-to-day C++ code, prefer `std::array` for fixed-size collections and `std::vector` when the size can change.

9. Common Pitfalls & Best Practices

9.1 Pitfalls to Avoid

- **Off-by-one errors** — looping with `i <= size` instead of `i < size` reads one element past the end.
- **Uninitialized arrays** — a locally declared array like `int a[5];` contains garbage values until you set them.
- **Array decay in functions** — losing size information when passing arrays; always pass the length too.
- **Assuming arrays can be resized** — they cannot; use `std::vector` when you need a growable container.
- **Comparing arrays with `==`** — this compares memory addresses (pointers), not contents. Use a loop or `std::equal` instead.

9.2 Best Practices

- Always initialize arrays before using them.
- Use named constants for sizes (`const int SIZE = 10;`) instead of "magic numbers".
- Prefer range-based `for` loops when you don't need the index.
- Validate indices before accessing elements when the index comes from user input.
- Prefer `std::array` / `std::vector` in modern code for safety and convenience.

10. 10 Practical Examples — Full Syntax & Explanation

Each example below is a complete, compilable C++ program. Read the explanation first, then study the code and its expected output. These examples move from beginner basics to slightly more advanced, commonly tested array operations.

Example 1 — Declaring, Initializing & Printing an Array

Goal: Learn the most basic operation — creating an array and printing every value using a `for` loop.

```cpp
#include <iostream>
using namespace std;

int main() {
    int numbers[5] = {10, 20, 30, 40, 50};

    cout << "Array elements: ";
    for (int i = 0; i < 5; i++) {
        cout << numbers[i] << " ";
    }
    cout << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Array elements: 10 20 30 40 50
```

Explanation: `numbers[5]` creates 5 slots (index 0 to 4). The `for` loop starts at `i = 0` and stops when `i` reaches 5, printing each element along the way.

Example 2 — Sum and Average of Array Elements

Goal: Accumulate values from an array — a pattern used constantly in real programs (totals, statistics, billing systems).

```cpp
#include <iostream>
using namespace std;

int main() {
    int scores[6] = {80, 92, 75, 68, 90, 88};
    int sum = 0;

    for (int i = 0; i < 6; i++) {
        sum += scores[i];
    }

    double average = static_cast<double>(sum) / 6;

    cout << "Sum = " << sum << endl;
    cout << "Average = " << average << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Sum = 493
Average = 82.1667
```

Explanation: `sum` starts at 0 and each loop iteration adds the next element with `+=`. `static_cast<double>` converts the integer sum before dividing, ensuring a decimal (not truncated integer) average.

Example 3 — Finding the Maximum and Minimum Value

Goal: Learn the "keep a running best value" pattern used in ranking, grading, and optimization problems.

```cpp
#include <iostream>
using namespace std;

int main() {
    int temps[7] = {21, 19, 25, 30, 18, 27, 23};

    int maxVal = temps[0];
    int minVal = temps[0];

    for (int i = 1; i < 7; i++) {
        if (temps[i] > maxVal) maxVal = temps[i];
        if (temps[i] < minVal) minVal = temps[i];
    }

    cout << "Max temperature = " << maxVal << endl;
    cout << "Min temperature = " << minVal << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Max temperature = 30
Min temperature = 18
```

Explanation: We assume the first element is both the max and min, then compare every other element and update whenever we find something bigger or smaller.

Example 4 — Reversing an Array In-Place

Goal: Learn the two-pointer swapping technique, used in reversing, palindrome checks, and many array-manipulation interview questions.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[5] = {1, 2, 3, 4, 5};
    int start = 0;
    int end = 4;   // last valid index

    while (start < end) {
        int temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }

    cout << "Reversed array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Reversed array: 5 4 3 2 1
```

Explanation: Two markers, `start` and `end`, swap elements and move toward each other until they meet in the middle — reversing the array without needing a second array.

Example 5 — Linear Search

Goal: Search for a target value by checking every element one by one — works on any array, sorted or not.

```cpp
#include <iostream>
using namespace std;

int linearSearch(int arr[], int size, int target) {
    for (int i = 0; i < size; i++) {
        if (arr[i] == target) {
            return i;  // found: return index
        }
    }
    return -1;  // not found
}

int main() {
    int data[6] = {4, 8, 15, 16, 23, 42};

    int pos = linearSearch(data, 6, 16);
    if (pos != -1)
        cout << "Found 16 at index " << pos << endl;
    else
        cout << "Not found" << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Found 16 at index 3
```

Explanation: The function checks each element against the target. If it matches, it returns that index immediately; if the loop finishes without a match, it returns `-1` to signal "not found". Time complexity: O(n).

Example 6 — Binary Search (on a Sorted Array)

Goal: A much faster search for sorted arrays — repeatedly cuts the search space in half.

```cpp
#include <iostream>
using namespace std;

int binarySearch(int arr[], int size, int target) {
    int low = 0, high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (arr[mid] == target) return mid;
        else if (arr[mid] < target) low = mid + 1;
        else high = mid - 1;
    }
    return -1;
}

int main() {
    int sorted[7] = {2, 5, 8, 12, 16, 23, 38};

    int pos = binarySearch(sorted, 7, 23);
    cout << "23 found at index: " << pos << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
23 found at index: 5
```

Explanation: The array MUST be sorted first. Each step compares the middle element to the target and discards the half that cannot contain it, giving O(log n) performance — dramatically faster than linear search for large arrays.

Example 7 — Bubble Sort (Sorting an Array)

Goal: Understand the classic beginner sorting algorithm by repeatedly comparing and swapping adjacent elements.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[6] = {64, 25, 12, 22, 11, 90};
    int n = 6;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }

    cout << "Sorted array: ";
    for (int x : arr) cout << x << " ";
    cout << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Sorted array: 11 12 22 25 64 90
```

Explanation: The outer loop controls the number of passes; the inner loop compares neighboring elements and swaps them if they are out of order. After each pass, the largest remaining element "bubbles" to its correct position at the end.

Example 8 — 2D Array: Matrix Addition

Goal: Apply array concepts to two dimensions — a direct real-world use case in graphics, simulations, and data tables.

```cpp
#include <iostream>
using namespace std;

int main() {
    int a[2][2] = {{1, 2}, {3, 4}};
    int b[2][2] = {{5, 6}, {7, 8}};
    int result[2][2];

    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            result[i][j] = a[i][j] + b[i][j];
        }
    }

    cout << "Resulting matrix:" << endl;
    for (int i = 0; i < 2; i++) {
        for (int j = 0; j < 2; j++) {
            cout << result[i][j] << " ";
        }
        cout << endl;
    }

    return 0;
}
```

EXPECTED OUTPUT

```
Resulting matrix:
6 8
10 12
```

Explanation: Nested loops walk through every row and column. Each cell of `result` is the sum of the matching cells from `a` and `b` — the same logic used for image blending and grid-based calculations.

Example 9 — Passing Arrays to Functions (with a Size Parameter)

Goal: Write reusable, modular code by moving array logic into its own function — an essential real-world coding habit.

```cpp
#include <iostream>
using namespace std;

// Doubles every element of the array (modifies the original)
void doubleValues(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = arr[i] * 2;
    }
}

void printArr(int arr[], int size) {
    for (int i = 0; i < size; i++) cout << arr[i] << " ";
    cout << endl;
}

int main() {
    int nums[4] = {3, 6, 9, 12};

    cout << "Before: "; printArr(nums, 4);
    doubleValues(nums, 4);
    cout << "After:  "; printArr(nums, 4);

    return 0;
}
```

EXPECTED OUTPUT

```
Before: 3 6 9 12
After:  6 12 18 24
```

Explanation: Because arrays "decay" to pointers when passed to a function, changes made inside `doubleValues` directly affect the original `nums` array in `main` — no copy is made, and no return value is needed.

Example 10 — Modern C++: std::array with STL Algorithms

Goal: See how modern C++ improves on raw arrays with `std::array`, which knows its own size and works seamlessly with the Standard Library.

```cpp
#include <iostream>
#include <array>
#include <algorithm>   // for sort, max_element
using namespace std;

int main() {
    array<int, 5> nums = {40, 10, 30, 20, 50};

    sort(nums.begin(), nums.end());   // sort ascending

    cout << "Sorted: ";
    for (int n : nums) cout << n << " ";
    cout << endl;

    cout << "Size: " << nums.size() << endl;
    cout << "Largest: " << *max_element(nums.begin(), nums.end()) << endl;

    return 0;
}
```

EXPECTED OUTPUT

```
Sorted: 10 20 30 40 50
Size: 5
Largest: 50
```

Explanation: `std::array<int, 5>` behaves like a C-style array but adds useful member functions like `.size()`, plus full compatibility with STL algorithms such as `sort()` and `max_element()` — safer and more convenient than raw arrays, with no loss of performance.

11. Quick Reference Cheat Sheet

Operation	Syntax
Declare	`int arr[5];`
Declare + Initialize	`int arr[5] = {1,2,3,4,5};`
Auto-sized	`int arr[] = {1,2,3};`
Access element	`arr[i]`
Modify element	`arr[i] = value;`
Array length (compile-time)	`sizeof(arr)/sizeof(arr[0])`
Traverse (classic)	`for(int i=0;i<n;i++)`
Traverse (range-based)	`for(int x : arr)`
2D declare	`int m[rows][cols];`
Pass to function	`void f(int arr[], int size)`
Modern fixed array	`std::array<int,5> a;`
Dynamic array	`std::vector<int> v;`

Final Note for Beginners

Master these fundamentals with plain C-style arrays first — they build a true understanding of memory, indexing, and pointers. Once comfortable, transition to `std::array` and `std::vector` for safer, more expressive everyday C++ code.